ALUMNICONNECT

APPLICATION ARCHITECTURE

Frontend, backend modules and database interactions

A technical design document for the Alumni–Student Mentorship and Mock Interview Platform

FRONTEND	BACKEND	DATA
Next.js + TypeScript	FastAPI modular monolith	MySQL + Redis + Neo4j

Prepared from the supplied DBMS Project Blueprint and course materials

Version 1.0 | 02 November 2026

1. Purpose and architectural direction

This document defines how the AlumniConnect web application is divided into user-facing interfaces, backend business modules and database systems. It translates the supplied project blueprint into an implementation-oriented architecture that can be built and demonstrated as a DBMS mini-project.

1.1 Product boundary

AlumniConnect connects college students with alumni who can provide career guidance, mentorship and mock interviews. Students maintain profiles, discover suitable mentors, submit requests, book appointments and review completed sessions. Alumni maintain professional profiles, publish availability, accept requests and record session outcomes. Administrators manage users, reference data and platform-level reports.

1.2 Core architectural decision

Recommended architecture: modular monolith + polyglot persistence

Deploy one FastAPI backend application. Organise it into independent business modules, but keep coordinated business operations in one application and one authoritative MySQL database. Use Redis and Neo4j only for their specialised access patterns. The API boundary of FastAPI is the logical API Gateway; it does not need to be a separately deployed gateway service.

1.3 Why this is the right fit

- Booking, mentorship allocation and session updates involve related records that should be committed together in SQL transactions.
- A database-per-microservice design would add distributed consistency and deployment work without a project-scale benefit.
- Clear module boundaries preserve separation of responsibilities and make the backend easier to test and extend.
- The design still demonstrates relational SQL, ACID transactions, caching, key-value data structures and graph traversal.

1.4 Design principles

Principle	Meaning in AlumniConnect
Single source of truth	MySQL owns identities, profiles, requests, allocations, availability, bookings, sessions, feedback and audit/notification records.
Specialised persistence	Redis accelerates reads and stores temporary state; Neo4j supports relationship-oriented recommendations; vector search is optional.
Business logic in modules	Modules own domain rules and coordinate repository/database access; routers should remain thin.
Transactional correctness	Critical state changes are validated and committed atomically in MySQL/InnoDB.
Asynchronous side effects	Outbox events drive notifications, graph projection updates and cache invalidation after the MySQL commit.

Important distinction

Redis and Neo4j are not replicas that can overrule MySQL. Redis values are disposable/rebuildable. Neo4j is an eventually consistent graph projection. MySQL is authoritative whenever data disagrees.

2. Overall application architecture

The application follows the three-tier client-server pattern described in the DBMS Unit 1 material: a presentation tier, an application tier and a data tier. The browser never connects directly to MySQL, Redis or Neo4j. All data access passes through the FastAPI application boundary.

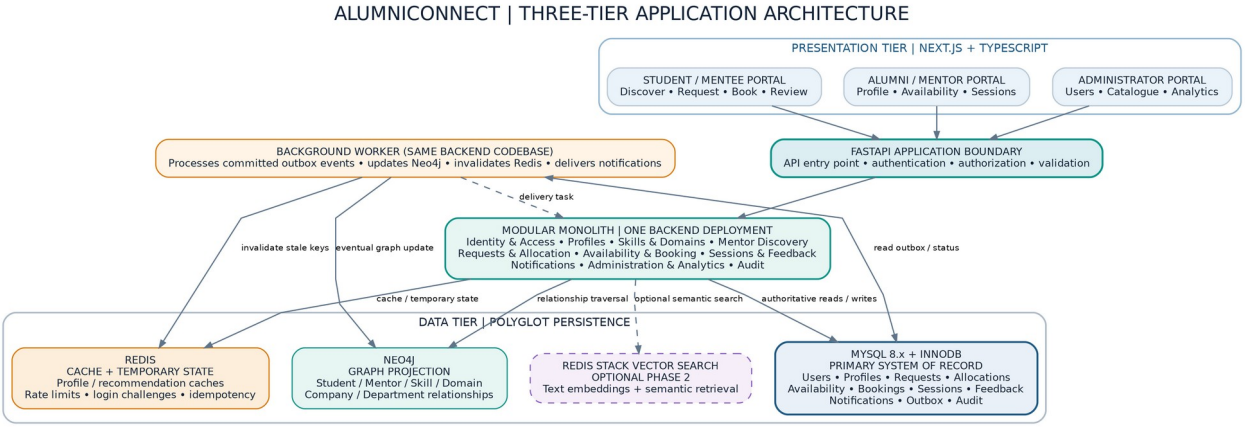


Figure 1. Proposed high-level architecture. Solid paths represent normal request/data flow; dashed paths represent optional or asynchronous work.

2.1 Tier responsibilities

Tier	Components	Responsibility
Presentation tier	Next.js + TypeScript	Role-based web experience, forms, navigation, client-side validation, tables, dashboards and API calls.
Application tier	FastAPI modular monolith	Single API boundary; authentication and authorization; request validation; business rules; transaction orchestration; database adapters.
Data tier	MySQL / Redis / Neo4j	MySQL stores authoritative relational records; Redis stores cache and short-lived state; Neo4j stores graph-shaped relationship projections.
Background execution	Outbox worker in the backend codebase	Reads committed outbox events and performs retryable notification delivery, Neo4j updates and Redis invalidation.

2.2 API Gateway interpretation

The API Gateway is a logical boundary, not a required additional service. In this design, the FastAPI application exposes the public API and performs routing, authentication, authorization, input validation and response formatting. Internal modules are called by the routers/application services; the frontend does not call database servers directly.

2.3 Request path at a glance

1. A user performs an action in the Student, Mentor or Administrator portal.
2. Next.js sends an HTTPS request to a FastAPI endpoint with the user identity/session context.
3. The API boundary authenticates the caller and checks the role and resource-level permission.
4. The responsible backend module applies business rules and chooses the required database access pattern.
5. The API returns a response; asynchronous work is processed from the outbox after the main SQL transaction commits.

3. Frontend architecture

The frontend is a single Next.js + TypeScript application with role-aware routes and reusable UI components. Student, alumni and administrator experiences are separate product areas inside the same frontend; they are not three independent deployments.

3.1 Portal structure

Portal	Main screens and actions	Data typically requested
Student / Mentee	Dashboard; edit profile; skills and interests; mentor discovery; recommendation details; submit mentorship request; view request status; browse available slots; create/cancel booking; session history; submit feedback and mentor review.	Own account/profile; skills/domains; mentor cards; recommendation evidence; request and allocation state; availability previews; bookings; session outcomes; feedback status.
Alumni / Mentor	Dashboard; professional profile; skills and domains; company and experience; availability calendar; incoming requests; accept/reject actions; booking calendar; session recording; feedback history.	Own profile; mentor skills/domains; pending requests; active allocations; availability; bookings; session records; received feedback and reviews.
Administrator	User management; role/status controls; department/company/skill/domain catalogue; platform overview; request/allocation/session reports; audit history.	User and profile summaries; reference data; aggregate metrics; status histories; audit entries. Private profile fields are not exposed unnecessarily.

3.2 Suggested frontend route map

Route group	Illustrative routes	Access
Public	/login, /register, /forgot-password	Unauthenticated entry and account recovery.
Student	/student/dashboard, /student/profile, /student/mentors, /student/requests, /student/bookings, /student/sessions	Student role; resource ownership checks remain on the backend.
Mentor	/mentor/dashboard, /mentor/profile, /mentor/availability, /mentor/requests, /mentor/sessions	Mentor role; only assigned requests and own schedule are accessible.
Admin	/admin/users, /admin/catalogue, /admin/reports, /admin/audit	Administrator role and permission checks.
Shared	/notifications, /settings, /profile	Shared shell; data and available actions depend on role.

3.3 Frontend component organisation

- Application shell: navigation, top bar, role-aware menu, notification indicator and account menu.
- Feature areas: identity, profiles, skills/domains, discovery, requests, scheduling, sessions, feedback and administration.
- Shared components: mentor card, skill chip, domain selector, status badge, availability calendar, booking summary, feedback form and data table.
- API client: a central typed client for base URL, auth/session context, request serialization, error handling and response types.
- UI state: keep transient form and view state in the frontend; reload authoritative booking/request status from the API rather than trusting stale client state.

> **Frontend security rule**
>
> Hiding a button or route is a user-experience measure, not authorization. FastAPI must independently verify every protected action and check whether the caller owns or participates in the referenced record.

4. Backend architecture and module boundaries

FastAPI exposes endpoint routers and delegates actual work to application services. Each module owns one business capability and its rules. Modules can share the same MySQL transaction/session when a workflow crosses domain boundaries.

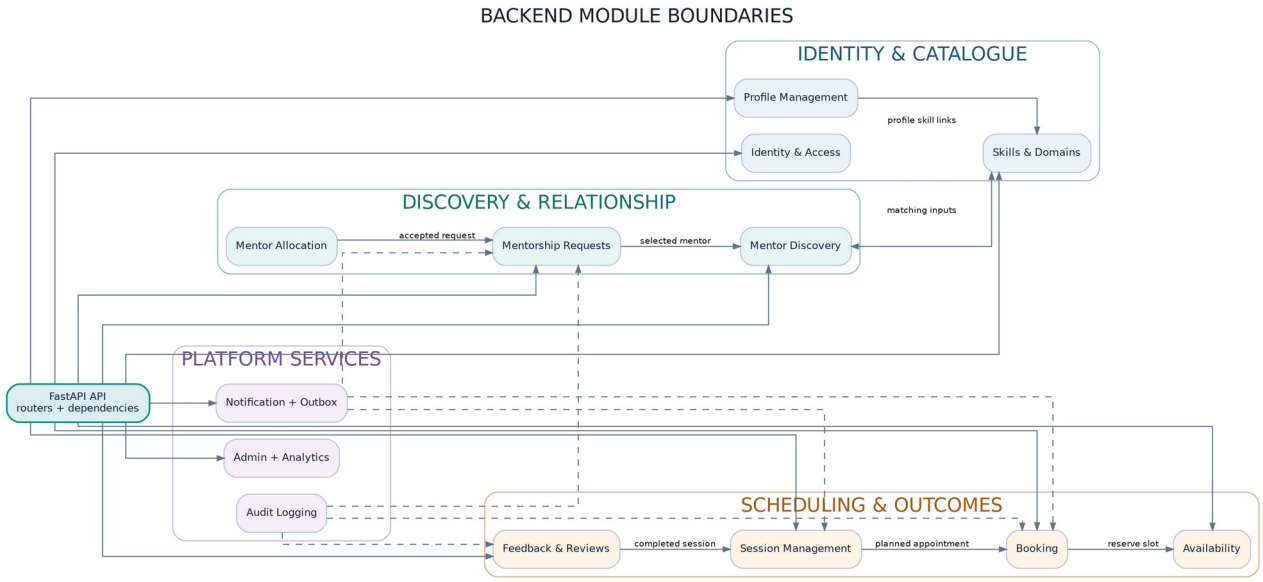


Figure 2. Logical module boundaries inside one FastAPI deployment. The arrows indicate important domain dependencies, not separate network services.

4.1 Recommended internal layering

Internal layer	What belongs here	What should not happen here
API / routers	HTTP routes, authentication dependencies, request/response schemas, status codes.	No long business workflows or direct raw database logic spread across route handlers.
Application services	Use-case orchestration such as accept request, create booking, generate recommendations and complete session.	No UI rendering or direct browser concerns.
Domain rules	Valid state transitions, role/participant rules, capacity checks and eligibility rules.	No assumptions that Redis or Neo4j are authoritative.
Repositories / adapters	MySQL queries and transactions; Redis cache operations; Neo4j queries; notification delivery adapter.	No business decisions hidden in generic cache/repository helpers.
Background worker	Outbox polling, retry logic, event dispatch, graph projection and cache invalidation.	No independent direct writes to MySQL and Neo4j as an uncoordinated dual-write.

4.2 Module catalogue

Module	Primary responsibilities	Main dependencies
Identity & Access	Registration, login, identity, roles, permissions and account status.	MySQL users/roles; Redis temporary login challenges, session support and rate limits.
Profile Management	Student and mentor profile read/update; professional biography, education, company and experience.	MySQL profiles, department/company; Redis profile cache; outbox on changes.
Skill & Domain Management	Maintain skills, interests, domains, proficiency and expertise associations.	MySQL catalogue and junction records; Neo4j projection; Redis catalogue cache.
Mentor Discovery	Search, match and rank mentors; return explainable reasons.	Neo4j traversal; MySQL eligibility and rating/capacity checks; Redis result cache; optional vector search.
Mentorship Requests	Create, list, cancel, accept and reject requests; maintain state history.	MySQL requests and allocation; outbox notifications; audit records.
Mentor Allocation	Create the accepted student-mentor relationship; prevent duplicate active allocations.	MySQL request/allocation transaction; capacity and active-state checks.
Availability Management	Create, update and withdraw mentor slots; show availability previews.	MySQL availability; Redis may cache non-authoritative display data only.
Booking Management	Validate eligibility, reserve a slot, prevent duplicate/overlapping booking and process cancellation.	MySQL/InnoDB transaction, locks and constraints; Redis idempotency record.
Session Management	Create or maintain planned session record; record completed/cancelled/no-show outcome and notes.	MySQL booking/session; outbox reminders and event history.
Feedback & Reviews	Capture structured participant feedback and student review of mentor.	MySQL completed session, feedback and review; aggregate ratings.
Notification Management	Create notification records and deliver request/booking/session/feedback notifications.	MySQL outbox and notification; background worker; delivery adapter.
Administration & Analytics	Manage users and reference data; generate SQL reports and audit views.	MySQL views, joins, aggregates, CTEs, windows and audit log.

5. How modules collaborate

A module boundary separates responsibility, but a complete user action often crosses several modules. The orchestrating application service coordinates the sequence and defines where the transaction begins and ends.

5.1 Student requests a mentor

1. Student Discovery page sends skill, domain, experience and career-goal filters to the Mentor Discovery API.
2. Discovery uses Neo4j for relationship-based candidate generation and MySQL for current eligibility, capacity and quality checks. Redis may serve a valid cached recommendation result.
3. The frontend displays ranked mentor cards with reasons such as shared skills, domain relevance and relevant industry experience.
4. When the student selects a mentor, the Mentorship Requests module re-validates the selected mentor and creates a Pending request in MySQL.
5. The same SQL transaction records an outbox event. After commit, the worker delivers a request notification to the mentor.

> **Request states**
>
> Pending -> Accepted / Rejected / Cancelled. Rejected and cancelled requests remain in history for reporting; they are not deleted merely because the request did not lead to an allocation.

5.2 Mentor accepts or rejects a request

1. The mentor opens the pending request and submits an accept/reject action.
2. The backend verifies that the request is still Pending, that the caller is the assigned mentor and that the mentor can accept another allocation.
3. On acceptance, MySQL creates the allocation and changes the request status inside one transaction; constraints and duplicate-active-allocation checks protect integrity.
4. An outbox record is inserted in the same transaction. After commit, the student receives the outcome notification.
5. On rejection, the request status is updated and retained in history; no allocation is created.

5.3 Availability and bookings

Availability is published by the mentor and stored in MySQL. The student interface can display a cached or previously fetched availability view, but the booking command always re-reads and validates the current slot in MySQL before confirming it.

5.4 Offline session recording

A booking is the planned appointment; a mentorship session is the actual meeting and its outcome. The mentor records whether the meeting was completed, cancelled or marked as no-show, plus discussion topics, skills assessed, observations, improvement areas and next steps. The system preserves the history of the session.

5.5 Feedback and mentor review

Session feedback captures structured comments from an actual participant (for example communication, helpfulness, preparation and notes). A mentor review is a student rating/review of the mentor after a completed session. These are separate concepts because they answer different reporting questions.

6. MySQL: authoritative relational database

MySQL 8.x with InnoDB is the system of record. It is responsible for relational integrity, foreign keys, uniqueness, transaction boundaries and the authoritative state of every business workflow.

6.1 Logical data domains

Domain	Core entities / tables (conceptual)	Why they exist
Identity & profiles	User, Role, Student Profile, Mentor/Alumni Profile, Department, Company	One authentication identity; separate role-specific information; controlled reference data.
Skills & expertise	Skill, Student Skill, Mentor Skill, Mentorship Domain, Student Domain Interest, Mentor Domain Expertise	Many-to-many links between people, capabilities and career domains.
Discovery history	Recommendation Run, Recommended Mentor / Candidate Recommendation	Preserve which candidates were suggested, their scores and the reason for a recommendation.
Request & allocation	Mentorship Request, Mentor Allocation	Separate a request for help from the accepted, active mentorship relationship.
Scheduling & sessions	Mentor Availability, Booking, Mentorship Session	Separate publishable time slots, planned appointments and actual session outcomes.
Feedback & communication	Session Feedback, Mentor Review, Notification, Notification Outbox, Audit Log	Capture structured experience, mentor rating, reliable notifications and important changes.

6.2 Key conceptual relationships

Relationship	Cardinality	Design interpretation
User -> Student Profile	1 : 0..1	An account may have one student profile.
User -> Mentor Profile	1 : 0..1	An account may have one mentor profile.
Student <-> Skill	M:N	A junction relation stores student-skill membership and proficiency where needed.
Mentor <-> Skill	M:N	A junction relation stores mentor expertise and experience/proficiency metadata.
Student -> Mentorship Request	1:N	A student can submit multiple requests; status is retained.
Request -> Allocation	1 : 0..1	One request produces at most one allocation.
Allocation -> Session	1:N	An active mentorship relationship can produce multiple sessions.
Mentor -> Availability	1:N	A mentor can publish multiple available slots.
Availability -> Booking	1 : 0..1	A slot can be booked at most once.
Booking -> Session	1 : 0..1	A confirmed booking can produce a session record.

6.3 Core integrity rules

- Use primary keys to identify each record and foreign keys to protect relationships between accounts, profiles, requests, allocations, bookings, sessions and feedback.
- Use unique constraints to prevent duplicate identities and duplicate ownership of a single availability slot.
- Use status constraints / controlled application transitions so requests and sessions cannot jump to invalid states.
- Only participants can access or submit session feedback; one student can submit at most one mentor review per session.
- A review must reference a valid completed session; feedback must not be accepted before the meeting has occurred.
- Retain rejected/cancelled request history and session history rather than deleting business history.

Schema note

The entities above describe the proposed conceptual model. Exact column names, data types, check constraints, indexes and physical table decomposition should be finalised during relational schema design and normalization.

7. Redis and Neo4j: specialised data stores

7.1 Redis - performance and temporary state

Redis is a performance layer, not the source of truth. It reduces repeated reads for stable or frequently requested information and supports short-lived platform state.

Feature	Suggested Redis structure	Purpose / policy
Mentor profile cache	Hash	Frequently viewed profile card fields; cache-aside; invalidate after profile change.
Skill and domain catalogue	Hash / Set	Quickly load stable reference values; use a bounded TTL or explicit invalidation.
Recommendation result cache	String / Hash	Reuse recent computed recommendation results; invalidate when relevant skills/profile data changes.
API rate limiting	String / counter	Short time-window counters to control repeated requests.
Temporary login challenge	String	Short-lived verification state; expiry is mandatory.
Request idempotency	String	Short-lived key to prevent duplicate effects when a client retries a request.
Popular mentor ranking	Sorted Set	Fast retrieval of a precomputed ranking view; source metrics remain in MySQL.
Recent activity feed	List / Stream	Temporary activity display; authoritative business history remains in MySQL.

Cache-aside read pattern

1. The API checks Redis for the requested profile or recommendation key.
2. On a cache hit, the application returns the cached display data after normal authorization checks.
3. On a cache miss, the application reads MySQL (or invokes the appropriate discovery path), builds the response and writes a cache entry with a TTL.
4. When the authoritative profile or skill changes, the update path invalidates relevant profile and recommendation keys. TTL is a safety net, not the only invalidation plan.

> **Never cache booking authority**
>
> Do not use Redis to decide whether a slot is free. Do not let a cache hit bypass MySQL authorization, availability validation, overlap checks or booking transaction logic.

7.2 Neo4j - relationship recommendation model

Neo4j stores a graph projection designed for traversal, not a second authoritative copy of relational business records. It helps discover and explain paths connecting students, mentors, skills, career domains, companies and departments.

Node label	Example relationship	Target node / useful properties
Student	HAS_SKILL	Skill; proficiency or learning level.
Mentor	HAS_SKILL	Skill; proficiency and experience information.
Student	INTERESTED_IN	Domain; current career interest.
Mentor	SPECIALIZES_IN	Domain; expertise area.
Mentor	WORKS_AT	Company; job title and employment period.
Student	STUDIES_IN	Department.
Mentor	ALUMNI_OF	Department.
Student	MENTORED_BY	Mentor; relationship status and mentoring history.

7.3 Recommendation dimensions

Dimension	Question answered
Skill similarity	How many relevant student skills overlap with mentor expertise?
Domain relevance	Does the mentor specialise in the student's preferred career domain?
Experience	How relevant and substantial is the mentor's industry experience?
Career relevance	Does the mentor's professional background fit the student's goal?
Mentor quality	What do historical ratings and outcomes indicate?
Capacity / eligibility	Can this mentor accept another student right now?

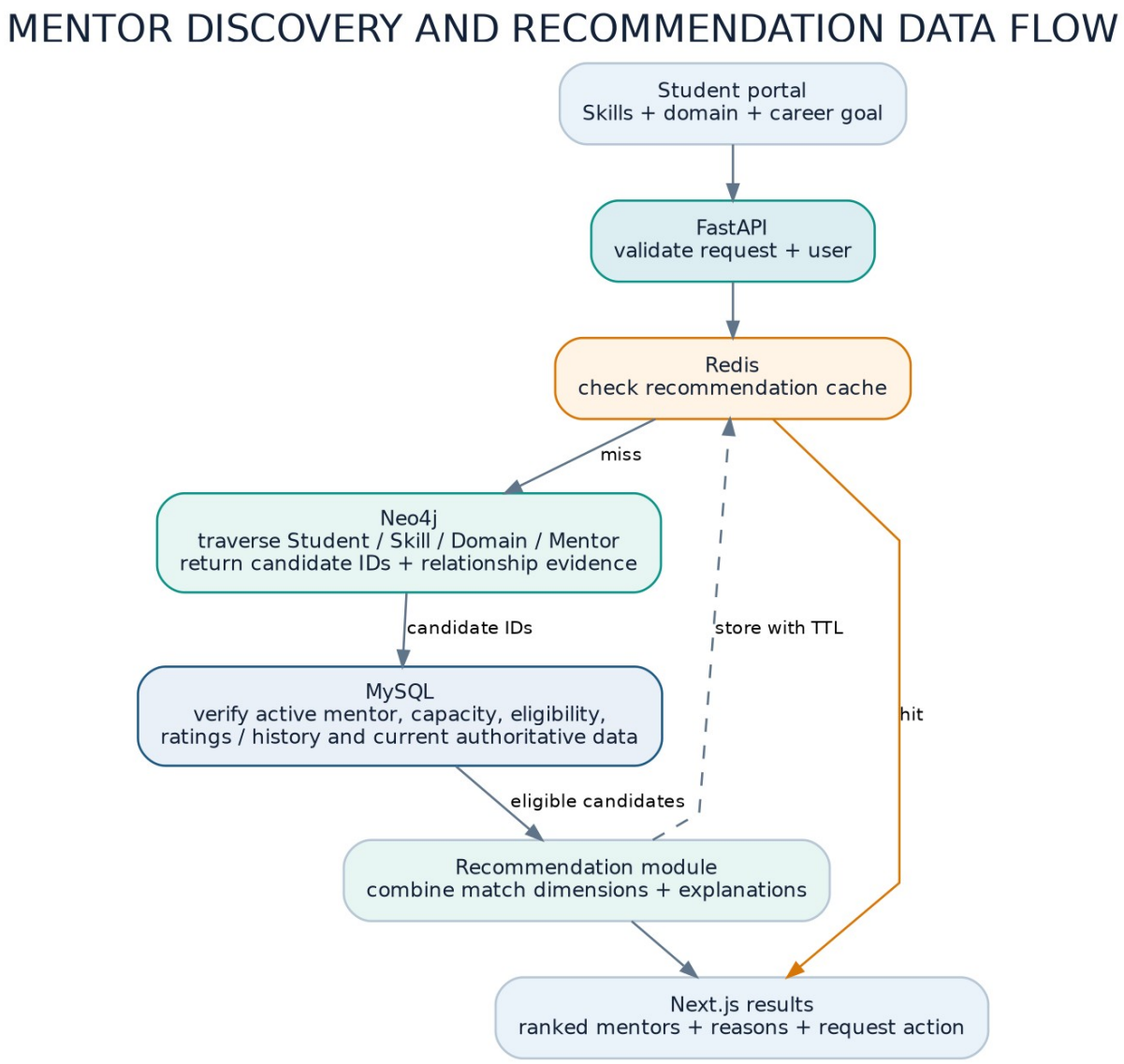


Figure 3. Recommendation flow: Neo4j finds relationship relevance; MySQL validates current eligibility; Redis caches a result when safe.

8. Cross-database consistency and event processing

The platform deliberately accepts different consistency roles for its stores: MySQL is strongly authoritative for business state; Redis is disposable cache/temporary state; Neo4j is an eventually consistent recommendation projection. This is safer than attempting an uncoordinated write to every database during a user request.

8.1 Transactional outbox pattern

When a MySQL business operation also requires a notification, graph update or cache invalidation, the application inserts an outbox event inside the same MySQL transaction as the business change. A worker reads committed events later and performs the secondary side effects.

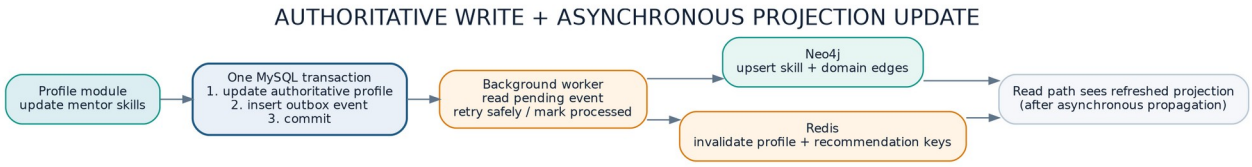


Figure 4. A profile update and its outbox event commit together. Secondary stores are updated asynchronously.

8.2 Example: mentor changes skills

Step	Action	Consistency consequence
1	Profile module validates the mentor and new skill selections.	MySQL remains the authoritative target.
2	One SQL transaction updates mentor-skill rows and inserts a MentorSkillsUpdated outbox event.	Either both changes commit, or neither does.
3	Background worker reads the committed event.	Worker can retry without losing the source event.
4	Worker upserts corresponding Neo4j nodes/relationships.	Graph projection catches up after the MySQL commit.
5	Worker invalidates profile and recommendation cache keys in Redis.	A later read reloads fresh data.
6	Worker marks the outbox event processed.	Delivery/projection status is observable.

8.3 Reliability requirements for the worker

- Process only committed outbox rows; do not publish a side effect before the SQL transaction commits.
- Use stable event identifiers and idempotent upsert/invalidation operations so retries do not create duplicates.
- Track pending, processed and failed attempts; retain enough error detail to investigate failures.
- Retry transient errors with bounded backoff; do not silently discard a failed event.
- Treat a delayed graph projection as a recommendation freshness issue, not as permission to skip MySQL validation.

Consistency rule

If Neo4j or Redis disagrees with MySQL, the MySQL result wins. A candidate returned by Neo4j may be removed by MySQL eligibility checks. A stale availability cache must never confirm a booking.

9. Booking workflow and concurrency control

Booking is the most critical consistency path because two students may attempt to reserve the same mentor slot at nearly the same time. A frontend availability display is only a preview. MySQL/InnoDB must make the final decision inside a transaction.

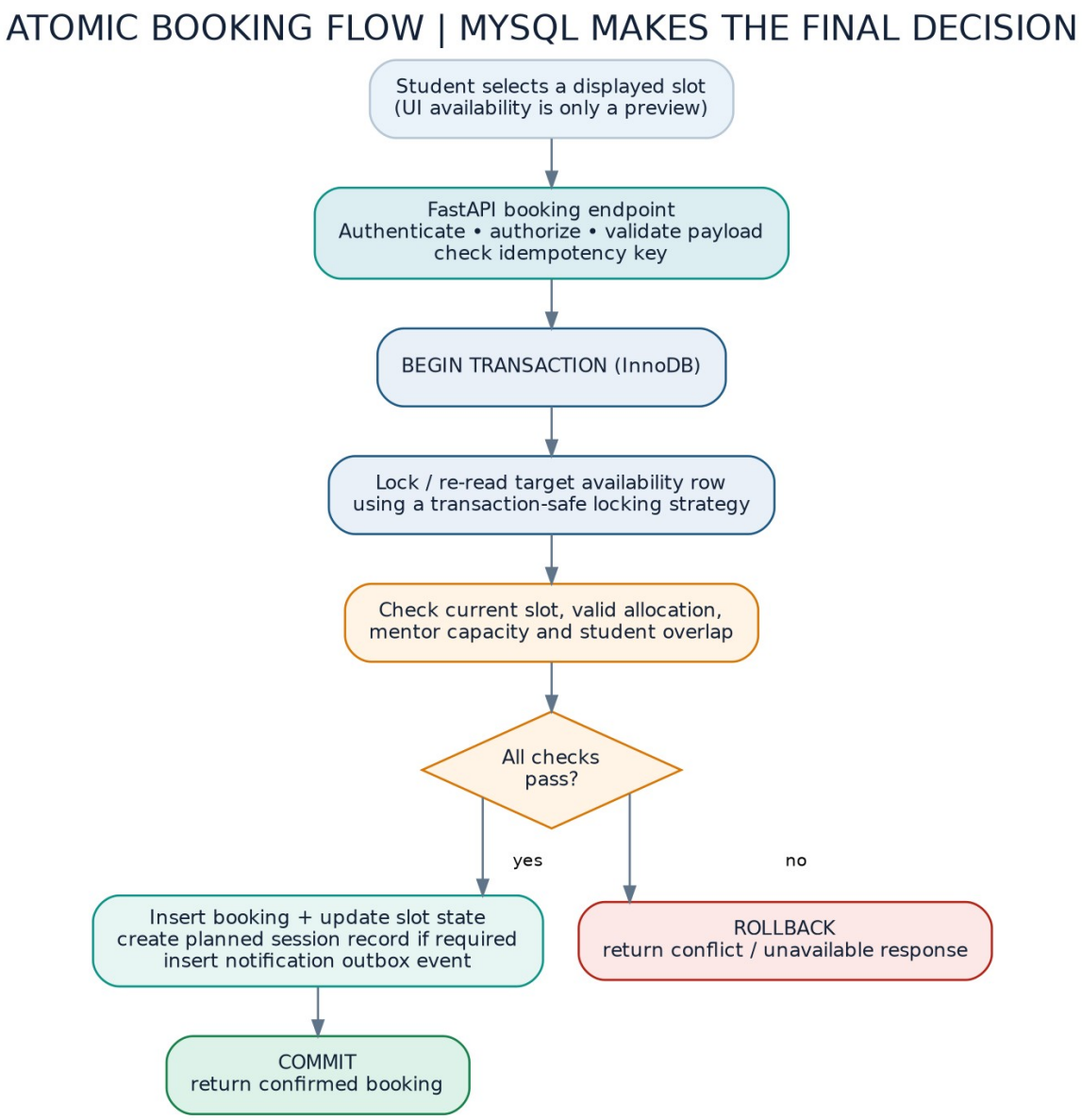


Figure 5. Conceptual atomic booking flow. The exact locking/query strategy is finalised with the physical schema and indexes.

9.1 Checks before confirmation

- The mentor availability slot still exists and is available at the time of the transaction.
- The student does not already have an overlapping confirmed booking.
- The mentor has not exceeded their mentorship or scheduling capacity.
- The referenced request or allocation is valid for the requested meeting.
- The submission is not a duplicate retry (idempotency key / uniqueness protection).

9.2 Transaction responsibilities

Transaction stage	Expected database action
Begin	Start one InnoDB transaction for the booking use case.
Read and lock	Re-read the target availability row using a transaction-safe locking strategy; do not rely on cached data.
Validate	Check slot state, allocation, mentor capacity, student overlap and duplicate submission.
Write	Insert the booking, update the slot state, create the planned session record if that is part of the chosen model, and insert the relevant outbox event.
Commit / rollback	Commit all related changes together. If a validation or write fails, roll back so no partial booking remains.
Respond	Return a confirmed booking or a conflict/unavailable response. The frontend refreshes the schedule from the API.

9.3 Concurrency safeguards

Use database uniqueness and transactional locking to prevent two requests from claiming the same slot. Also protect against overlapping appointments on the student side; locking only the mentor slot is not sufficient. The final schema should include the relevant unique keys and supporting indexes, and the booking service should test concurrent submissions under the selected InnoDB isolation/locking strategy.

> **Required concurrency demonstration**
>
> Run two simultaneous booking attempts for one slot. Exactly one must commit; the other must receive a conflict/unavailable result. Repeat with one student attempting two overlapping appointments.

10. Sessions, feedback and notifications

10.1 Booking is not the session

Record	Meaning	Typical state / fields
Booking	Planned appointment created when a student reserves a slot.	Time, mentor, student, availability slot, booking status, creation/cancellation details.
Mentorship Session	Actual offline meeting and its outcome.	Scheduled -> Completed / Cancelled / No-show; discussion topics, skills assessed, observations, areas to improve, next steps and preparation guidance.

Separating the two prevents a planned meeting from being mistaken for a completed mentoring interaction. A booking can be cancelled before it becomes a completed session, and the session record preserves what actually happened.

10.2 Feedback and review rules

- A session cannot receive feedback before it has taken place.
- Only the actual student and mentor participants can submit feedback for that session.
- One student can submit at most one mentor review per session.
- A mentor review must reference a valid completed session.
- Review history should not be silently overwritten; maintain auditable changes or append-only history where required.

10.3 Notification event matrix

Business event	Recipient	Where created / delivered
Mentorship request submitted	Mentor	Outbox event in the request transaction; worker creates/delivers notification.
Request accepted	Student	Outbox event in the acceptance transaction.
Request rejected	Student	Outbox event in the rejection transaction.
Booking confirmed	Student and mentor	Outbox event in the booking transaction.
Booking cancelled	Student and mentor	Outbox event in the cancellation transaction.
Session reminder	Student and mentor	Scheduled/background event; recipient and timing policy must be defined.
Feedback submitted	Relevant participant	Outbox event after valid feedback/review commit.

10.4 Notification processing sequence

1. A business module writes the domain change and outbox event in the same MySQL transaction.
2. After commit, a worker polls or claims pending outbox events.
3. The worker creates the notification record and invokes the configured delivery adapter.
4. Success/failure status is persisted so the task can be retried or inspected.
5. The user sees the notification through the web application; the authoritative notification record remains in MySQL.

11. API surface and module mapping

The following is a proposed REST-style endpoint catalogue that gives the frontend a stable contract. Route names are illustrative design recommendations; the supplied blueprint defines module responsibilities and workflows, not a finalized API specification.

Feature	Illustrative endpoint set	Responsible module
Identity	POST /api/v1/auth/register POST /api/v1/auth/login POST /api/v1/auth/logout GET /api/v1/me	Identity & Access
Profiles	GET /api/v1/students/me PATCH /api/v1/students/me GET /api/v1/mentors/{mentorId} PATCH /api/v1/mentors/me	Profile Management
Catalogue	GET /api/v1/skills GET /api/v1/domains GET /api/v1/departments GET /api/v1/companies	Skill & Domain Management
Discovery	GET /api/v1/mentors POST /api/v1/recommendations GET /api/v1/recommendations/{runId}	Mentor Discovery
Requests	POST /api/v1/mentorship-requests GET /api/v1/mentorship-requests POST /api/v1/mentorship-requests/{id}/accept POST /api/v1/mentorship-requests/{id}/reject POST /api/v1/mentorship-requests/{id}/cancel	Mentorship Requests / Allocation
Availability	POST /api/v1/mentors/me/availability GET /api/v1/mentors/{id}/availability DELETE /api/v1/availability/{slotId}	Availability Management
Bookings	POST /api/v1/bookings GET /api/v1/bookings POST /api/v1/bookings/{id}/cancel	Booking Management
Sessions	GET /api/v1/sessions PATCH /api/v1/sessions/{id}/outcome GET /api/v1/sessions/{id}	Session Management
Feedback	POST /api/v1/sessions/{id}/feedback POST /api/v1/sessions/{id}/mentor-review GET /api/v1/mentors/{id}/reviews	Feedback & Reviews
Notifications	GET /api/v1/notifications PATCH /api/v1/notifications/{id}/read	Notification Management
Administration	GET /api/v1/admin/users PATCH /api/v1/admin/users/{id} GET /api/v1/admin/reports/*	Administration & Analytics

11.1 Standard request handling

Concern	Recommended behaviour
Authentication	Resolve the current user from the authenticated request/session before executing protected operations.
Authorization	Check role plus ownership/participation. A mentor cannot accept another mentor's request; a student cannot review a session they did not attend.
Validation	Validate required fields, allowed status transitions, identifiers and time ranges before database writes.
Errors	Return clear 4xx responses for invalid, forbidden, missing, conflicting or unavailable operations; avoid exposing database internals.
Idempotency	For retry-sensitive create actions such as booking, accept an idempotency token and enforce duplicate protection in persistent state.
Versioning	Keep a versioned prefix such as /api/v1 so client and backend changes can be managed deliberately.

12. How the architecture demonstrates DBMS concepts

The design intentionally makes course concepts visible in real application work. It is not necessary to force every DBMS topic into every request; each concept should solve a real application problem.

DBMS unit / topic	Where it appears in the application	Concrete demonstration
Unit 1: Client-server and three-tier architecture	Next.js client -> FastAPI application server -> database tier.	Browser calls the application API; database servers are not exposed directly to the client.
Unit 1: ER model, relational model, keys and constraints	Identity, profiles, skills, requests, allocations, availability, bookings and sessions.	Explain entity relationships, junction tables, PK/FK, uniqueness and referential integrity.
Unit 1: SQL and relational operations	Profile pages, discovery filters and administration views.	Show SELECT/WHERE/JOIN and relational operations supporting actual screens.
Unit 2: Joins, nested queries and aggregates	Mentor search, rating reports, incomplete profile reports and request funnel.	Join mentor with company/skills/domain; calculate ratings; find unallocated students.
Unit 2: Views, triggers, procedures, CTEs, windows, full-text	Public mentor directory, audit trail, booking/acceptance operations, rankings and biography search.	Build public-safe views; audit status changes; rank mentors by domain; analyse funnel stages.
Unit 3: Functional dependencies and normalization	Core relational schema.	Identify candidate keys, partial/transitive dependencies and justify 3NF/BCNF decomposition.
Unit 3: Query processing and optimization	Mentor listing, session history and ranking queries.	Compare initial and optimized EXPLAIN plans; add relevant indexes and measure execution time.
Unit 4: Transactions, ACID and concurrency	Mentor acceptance, booking, session updates and outbox insertion.	Demonstrate atomic commit/rollback, isolation and simultaneous booking attempts.
Unit 4: Redis key-value database	Profile cache, recommendation cache, rate limits and temporary state.	Demonstrate hashes, sets, sorted sets, counters, TTL, cache hit/miss and invalidation.
Unit 4: Neo4j graph database	Mentor recommendation.	Create Student/Mentor/Skill/Domain nodes and traverse HAS_SKILL, INTERESTED_IN and SPECIALIZES_IN paths.
Unit 4: Vector search (optional)	Phase 2 natural-language mentor discovery.	Search semantically similar mentor descriptions and combine results with graph and SQL eligibility.

12.1 Query-processing path for a screen

For example, when the administrator opens a mentor analytics report, the FastAPI analytics service issues an SQL query. The DBMS scans and parses the SQL, validates referenced names, creates an internal query representation, chooses an execution plan through query optimization, and executes that plan to return the result. The application then formats the result for the dashboard.

> **Course alignment**
>
> The Unit 1 slides describe three-tier architecture as a client, an intermediate application/web server containing business logic and a database server accessible through the middle tier. The Unit 3 slides describe scanning, parsing, validation, query-tree/query-plan generation and query optimization. The proposed application architecture uses those ideas directly.

13. Security, reliability and operational boundaries

The blueprint establishes roles and permissions as part of Identity and Access. The following implementation controls are recommended to make that boundary reliable in a web application.

13.1 Access control

Control	Implementation responsibility
Role-based access	Assign Student, Mentor and Administrator roles; each API operation declares which roles may invoke it.
Resource ownership	Verify the caller owns the profile/booking/feedback record or is an actual participant in the relevant request/session.
Least privilege	Expose public mentor directory fields separately from private account and contact information.
Server-side enforcement	Repeat permission checks in FastAPI even when Next.js hides controls or redirects users.
Sensitive credentials	Store password hashes, never plaintext passwords; keep secrets and database credentials outside source code.

13.2 Reliability boundaries

Failure / risk	Expected behaviour
Redis unavailable	Continue with MySQL-backed reads where practical; cache loss must not corrupt business state.
Neo4j unavailable	Temporarily degrade discovery (for example, use supported SQL filters) or return a controlled unavailable response; do not invent eligibility.
Worker unavailable	Committed business updates remain valid; outbox events stay pending until processing resumes.
Duplicate client retry	Use idempotency/unique constraints so repeated submission does not create duplicate bookings or allocations.
Concurrent slot request	Only one transaction can claim the slot; losing request receives conflict/unavailable response.
Stale recommendation	Re-check current mentor eligibility and capacity in MySQL before a request/allocation/booking decision.

13.3 Observability

- Log request correlation ID, actor ID, module, operation outcome and duration; do not log passwords or secrets.
- Track booking conflicts, transaction rollbacks, cache hit/miss rate, recommendation latency and outbox backlog.
- Provide a way to inspect failed outbox events and retry them safely.
- Keep audit records for important account, request, allocation, booking, session and review status changes.

14. Deployment and runtime topology

The recommended deployment preserves the modular-monolith decision. Deploy one frontend application and one FastAPI backend application. Run MySQL, Redis and Neo4j as separate data services because they are distinct database engines; the background worker can run as a second process/container using the same backend codebase.

14.1 Logical runtime units

Runtime unit	Deployment responsibility	Configuration boundary
Next.js web app	Serves role-based web pages and static assets; calls FastAPI.	Public base URL; backend API base URL; browser-safe configuration only.
FastAPI API process	Handles incoming HTTP requests and orchestrates modules.	Private DB connections, secret keys, CORS/origin policy, allowed hosts and auth configuration.
Background worker	Consumes outbox rows and runs notifications/projection/cache tasks.	Same domain code and database adapters; worker-specific polling/retry settings.
MySQL 8.x / InnoDB	Persistent relational system of record.	Private network, backups, schema migrations and credentials.
Redis	Cache and temporary key-value state.	Private network, TTL policy, eviction policy and key namespace.
Neo4j	Graph recommendation projection.	Private connection credentials, indexes/constraints and projection rebuild procedure.
Optional Redis Stack vector capability	Embedding index and semantic candidate retrieval in Phase 2.	Separate feature flag and index/model configuration; no effect on core booking.

14.2 Practical development layout

For local development, the frontend, API and worker may run as local processes while MySQL, Redis and Neo4j run in local services or containers. A small team can develop against one shared integration contract without splitting every business module into a network service.

14.3 Suggested backend package layout

```
backend/
  app/
    main.py                # FastAPI application and API boundary
    api/v1/                # versioned routers and request schemas
    core/                  # configuration, auth, permissions, logging
    modules/
      identity/             # users, roles, authentication
      profiles/             # student and mentor profiles
      catalogue/            # skills, companies, departments, domains
      discovery/            # recommendation orchestration
      requests/             # mentorship requests and allocation
      scheduling/           # availability and booking
      sessions/             # session outcomes
      feedback/             # feedback and mentor reviews
      notifications/         # notification records and outbox
      administration/        # admin operations and reports
    infrastructure/         # MySQL, Redis, Neo4j and delivery adapters
    workers/                # outbox polling and event handlers
  migrations/               # relational schema changes
  tests/                    # module, API and integration tests
```

15. Implementation sequence and acceptance checklist

The safest build order is to establish the authoritative relational workflow first, then add the specialised stores. This delivers a working application early and avoids debugging multiple data systems before the core transactions are correct.

Stage	Build scope	Exit condition
1. Foundation	Next.js shell; FastAPI app; MySQL schema/migrations; User, Role and profile setup.	Student, Mentor and Admin can authenticate and access role-appropriate profile pages.
2. Catalogue and profiles	Departments, companies, skills, domains, student interests and mentor expertise.	Profiles and many-to-many skill/domain relationships persist correctly in MySQL.
3. Requests and allocation	Mentor search baseline; request lifecycle; mentor accept/reject; allocation rules.	Requests retain status history; acceptance and allocation are atomic and duplicate-safe.
4. Availability and booking	Mentor availability; booking transaction; cancellation; concurrency tests.	Two simultaneous requests for one slot cannot both commit; student overlap is also protected.
5. Sessions and feedback	Session outcomes; structured feedback; mentor review; rating aggregates.	Only valid participants can submit feedback; reviews require completed sessions.
6. Notifications and admin	Outbox worker; notification delivery; audit log; dashboard reports.	Notifications are emitted only from committed events; worker retries are safe.
7. Redis performance	Profile/catalogue/recommendation cache; rate limiting; TTL and invalidation.	Cache loss or stale values do not change authoritative business outcomes.
8. Neo4j recommendations	Graph projection; Cypher traversal; explainable candidate list; sync worker.	Graph generates relationship candidates; MySQL performs final eligibility checks.
9. Optional vector extension	Redis Stack vector index and semantic search.	Natural-language search improves discovery without bypassing graph/SQL validation.

15.1 Architecture acceptance checklist

- There is one FastAPI application boundary; backend capabilities are modules, not separately deployed microservices.
- The frontend never connects directly to MySQL, Redis or Neo4j.
- All authoritative identity, profile, request, allocation, scheduling, booking, session, feedback and audit records are in MySQL.
- Redis is used for caches/temporary state and never decides slot availability.
- Neo4j stores a relationship projection and can be rebuilt from authoritative data/events.
- Critical writes use MySQL transactions; booking has uniqueness, lock/check and rollback protection.
- Outbox events are inserted in the same SQL transaction as their business change and processed asynchronously.
- Feedback and review rules enforce completed-session and participant requirements.
- Recommendation results are explainable and are revalidated against current MySQL eligibility.
- Unit demonstrations include SQL/normalisation/optimization, ACID/concurrency, Redis operations and Neo4j traversal.

15.2 Scope boundary

Core release

Student, alumni and administrator portals; identity and profiles; skills/domains; Neo4j-based mentor discovery; requests and allocation; availability and booking; offline sessions; feedback and ratings; notifications; SQL analytics; Redis caching.

Phase 2 extension

Semantic mentor discovery using Redis Stack vector search. Add it after the SQL and graph recommendation paths are working. It should enrich candidate discovery, then pass results through the same graph and MySQL checks.

Appendix A. Source alignment and design assumptions

This document is grounded primarily in the supplied file "Alumni–Student Mentorship Platform: DBMS Project Blueprint". It preserves that document's key decisions: AlumniConnect scope, Student/Mentor/Admin roles, modular monolith, FastAPI, Next.js, MySQL, Redis, Neo4j, optional Redis Stack vector search, source-of-truth boundaries, graph projection, cache-aside, transactional outbox, atomic booking, session/feedback separation and four-unit DBMS mapping.

Provided material	Used in this document
Alumni–Student Mentorship Platform: DBMS Project Blueprint, pp. 1-5	Product scope, roles, core modules, modular-monolith decision, three-tier component choices and API-boundary interpretation.
Project Blueprint, pp. 6-9	MySQL source of truth; Redis performance use cases; Neo4j recommendation role; optional vector search and cross-store ownership.
Project Blueprint, pp. 10-13	Conceptual entities, cardinalities, common User identity, and separation of Booking from Mentorship Session.
Project Blueprint, pp. 14-20	Graph node/relationship model, matching dimensions, explainability and transactional-outbox synchronization.
Project Blueprint, pp. 20-27	Redis data structures and cache policy; request, allocation, booking, session, feedback and notification workflows.
Project Blueprint, pp. 28-34	DBMS unit mapping, SQL demonstration ideas, normalization/query optimization, concurrency tests and scope priorities.
DBMS Unit 1 slides, especially slides 28 and 31-33	Client-server, two-tier and three-tier application architecture; middle tier holds application connectivity and business logic.
DBMS Unit 3 slides, pp. 2-6	Query scanning, parsing, validation, query-tree/plan creation, optimization and execution.
DBMS Unit 4 slides, ACID and locking sections	Atomicity, consistency, isolation, durability and the need for concurrency control in booking workflows.

Appendix B. Explicit design assumptions

- The REST route catalogue, package layout, deployment units, security controls and operational metrics are proposed implementation details added to make the blueprint actionable; they are not claimed to be existing code or a finalized API contract.
- The exact booking lock strategy and the physical constraints needed to prevent time-interval overlap depend on the final schema, indexes and InnoDB isolation level. The architectural requirement is that MySQL performs the final serialized validation and commit.
- The notification delivery channel is left as an adapter because the supplied blueprint defines notification events and outbox processing but does not mandate a specific email/SMS/push provider.
- The graph projection is eventually consistent; recommendations can be slightly stale. Critical allocation and booking decisions must be re-validated in MySQL.
- Vector search is optional Phase 2 scope, not a prerequisite for the core application.

Appendix C. Short glossary

Term	Meaning in this design
Modular monolith	One deployable backend containing clearly separated business modules.
Polyglot persistence	Using multiple database types because each suits a different access pattern.
System of record	The authoritative database that owns the correct business value; MySQL here.
Cache-aside	Application checks cache first, reads the source on a miss and stores a temporary copy.
Projection	A derived representation built from authoritative records for a specialized query pattern.
Transactional outbox	A durable event row written in the same SQL transaction as the business update; a worker publishes it later.
Idempotency	Repeated execution of the same request produces no duplicate business effect.
Eventual consistency	A secondary store may lag briefly but converges after asynchronous updates are processed.